

Incentive Design for Heterogeneous Client Selection: A Robust Federated Learning Approach

Papa Pene^{1b}, Weixian Liao^{1b}, and Wei Yu^{1b}

Abstract—Federated learning (FL) allows the collaborative training of machine learning (ML) models between an aggregation server and different clients without sharing their private data. However, the FL archetype is mostly vulnerable to malicious model updates from various clients because of the privacy feature that makes the server see clients as a black box. When selecting clients, the server has no control on their contributions during training. This opacity of the server toward clients' data associated with the huge amount of heterogeneous data brings a security risk and poses a deterioration of the model performance in FL. The impact of client selection and data heterogeneity on FL robustness has been overlooked. In this article, we develop an incentive design for heterogeneous client selection (IHCS) to improve the performance while reducing the security risks in FL. The IHCS approach applies a smarter client selection method using cooperative game theory and dynamic clustering of clients based on their heterogeneity level to overcome the challenges of lacking access to clients' data, unbalanced data, and the lack of applicable data contribution from clients in FL. To do so, we attribute a recognition value to each client using the Shapley value. This recognition index is then used to aggregate the probability of participation level. We also implement, within the IHCS, a heterogeneity-based clustering (HIC) method that helps inhibit the negative influence of data heterogeneity and increase client contributions. Through extensive experiments with empirical results, the proposed approach outperforms the representative works on robustness of FL.

Index Terms—Data heterogeneity, distributed computing, federated learning (FL).

I. INTRODUCTION

FEDERATED Learning (FL) is a distributed machine learning (ML) paradigm that enables the collaboration of a server and a group of clients¹ to train models while preserving their local data privacy [1]. This distributed ML paradigm can be applicable to many Internet of Things (IoT) devices, which empower a variety of smart-world systems [2]. The rapid increase of IoT technology has brought significant changes to our daily lives. For instance, an emerging number of noncomputerized hospital devices have access to the Internet and doctors are using their smart-watches to read

Manuscript received 31 July 2023; accepted 21 August 2023. Date of publication 4 September 2023; date of current version 6 February 2024. This work was supported in part by the Air Force Office of Scientific Research under Award FA9550-20-1-0418. (Corresponding author: Weixian Liao.)

The authors are with the Department of Computer and Information Sciences, Towson University, Towson, MD 21252 USA (e-mail: ppene1@students.towson.edu; wliao@towson.edu; wyu@towson.edu).

Digital Object Identifier 10.1109/JIOT.2023.3311690

¹Note that the term client, agent, node, and user are used interchangeably in this article. In addition, the aggregator refers to the server in this article.

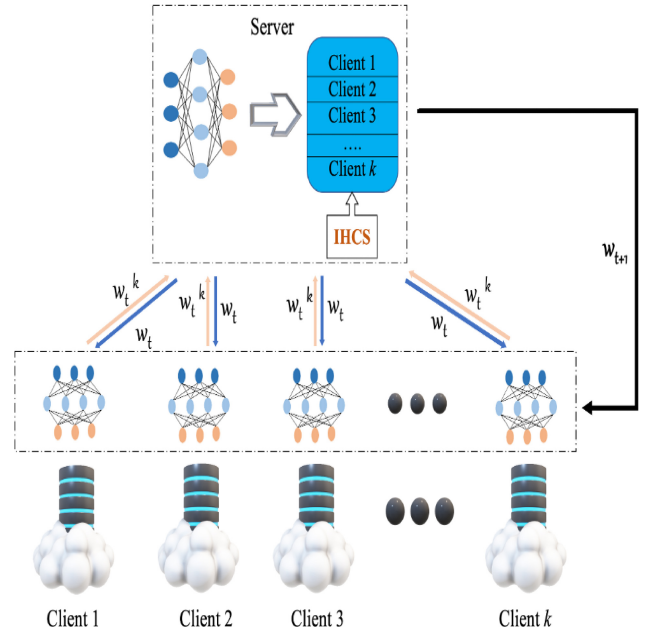


Fig. 1. FL overview with IHCS.

emails and access other medical documents that must comply with the health insurance portability and accountability act (HIPAA), etc. Now, FL deals with the matter of how to develop models involving the participation of IoT while keeping the balance between privacy and performance. All of this explains the disruptive changes brought on by federated learning (FL) into how to optimize products' quality alongside the client's privacy. Therefore, optimizing this decentralized distributed ML resource is important in the literature.

The optimization level of ML resources is unprecedented in the way that data is utilized to make machines think smarter [3], [4]. The general principle that FL works on is shown in Fig. 1, where at a time t , a central server distributes a model with a weight w_t to various clients k to update and send back the locally updated model with a weight w_t^k . Then, for the next round $t+1$, the server would send another updated model w_{t+1} until reaching an aggregated model, which would be used by clients without the need of sharing local data.

The two representative methods implemented in FL for an aggregated model are the federated stochastic gradient descent (FedSGD) and federated averaging (FedAvg) [1]. On the one hand, the FedSGD algorithm allows a single epoch of gradient descent while involving all clients at each round. On the other hand, in FedAvg, some clients are partially selected, and each

client can take multiple epochs to update a model. These two algorithms turn out to be effective but have shown their limit when it comes to robustness in FL. In fact, the dilemma for many researchers in FL has been improving the performance and security of models without a loss in privacy. Related to this, Geyer et al. [5] proposed a differential private method, which tends to the balance between privacy and performance in FL.

Nonetheless, FL is not a solution for all ML problems. In fact, FL faces numerous challenges that affect its performance and security. First, there is a lack of access to the clients' data. Privacy is essential in FL, yet it is also true that it presents itself a drawback. In FL, we witness an opacity between the aggregator and the users. This blindness from the decentralized server has ramifications on system security because it becomes vulnerable to poisoning attack such as backdoor. A backdoor attack in FL consists of inserting a feature in the model during a local update to lead the server to a misclassification [6]. Second, servers in FL face an issue of lack of client's useful data contribution. During training, some clients are not fully participating in the process but benefit from the aggregated model. When selecting clients, the server often runs into irrelevant or corrupted data since the server has no information about the type of data clients will contribute. Clients are labeled as irrelevant when the update shared by it can negatively impact the performance and the stability of the central model [7]. Third, data imbalance also poses a challenge to FL. Data imbalance issues are prevalent in FL. Often, the server may want to induce clients that possess data belonging to the minority class [7]. With the large implication of IoT devices, the wide variance of bandwidth, the disparity of the data type, etc., all clients cannot be treated the same way. The heterogeneous nature of various clients' data makes them have different contributions. FL might neglect the performance of most of the agents to fit with the performance of a smaller number of them with low applicable data [8]. These challenges have a negative impact on both the performance and the security of FL models. In this article, we propose an approach to mitigate them.

Although there are substantial studies on how to build robust frameworks, to the best of our knowledge, we have not seen a work that uses heterogeneous factors to reinforce robustness in FL. In fact, in many studies, data heterogeneity is identified as a major issue in FL. Most of the studies on FL were carried out in an independent and identical distribution (IID) setting. In an IID setting, it is assumed that the data samples (x_i, y_i) of a single partition D_k used to train the ML model w are independent and evenly spread among other data samples of the K partitions. On the other hand, in nonindependent and identical distribution (non-IID) setting, we do not make the same supposition, and the model w is not trained based on the (x_i, y_i) generated from a single D_k . Yet, the IID setting does not reflect reality and undermines heterogeneity. Actually, the non-IID setting is more reflective actual FL training since a client's local data set is not representative of the entire population. The existing works conducted in non-IID setting overlook the impact of data heterogeneity.

In this article, we propose a new approach, denoted as incentive design for heterogeneous client selection (IHCS), to improve robustness in FL. Our IHCS approach employs cooperative game theory to assign a value to client's participation using the concept of the Shapley value. The Shapley value is used to assign a recognition index of the gradient shared by each client and compute the probability of participation level before each round. Within the IHCS, a heterogeneity-based clustering (HIC) method is implemented. The HIC helps eliminate the negative impact of data heterogeneity to ameliorate the performance of the aggregated model. We calibrate client's heterogeneity level using a heterogeneity index formula [9]. Then, we make use of an affinity propagation (AP) technique to distribute a model's weight with a similar heterogeneity level for training. Consequently, we obtain a higher performance, lower attack success rate, and an increasing number of clients' useful participation in compared to existing FL schemes.

The main contributions of this article are synthesized as follows.

- 1) With the massive participation of IoT devices, our new scheme IHCS brings an intelligent client selection method that segregates model's weight by heterogeneity level.
- 2) The IHCS offers a framework that promotes client contribution using a participation scaling method that helps to remediate the lack of client participation.
- 3) The stability of the performance accuracy in the results proves that the IHCS contributes in the elimination of the presence of irrelevant data during training.

The remainder of this article is organized as follows. In Section II, we give a brief review of relevant research efforts related to our study. In Section III, we provide the work preliminary. In Section IV, we explain the IHCS approach in detail. In Section V, we evaluate the empirical results. In Section VI, we discuss some remaining issues. Finally, we conclude this article in Section VII.

II. RELATED WORK

In this section, we review the state-of-the-art literature related to our research from the aspects of robustness in FL, data heterogeneity, and cooperative game theory, respectively.

Robustness in FL: We define an FL framework's robustness as its ability to keep the integrity of its system, to withstand internal and external poisoning attacks, and to optimize the performance of its models. The FL paradigm focuses on privacy and performance, which have made it very attractive to a variety of systems. Substantial research efforts have been made to reinforce this type of ML paradigm and make it more enticing to industry applications. The primary development of FL focuses on building a robust aggregation method that assures the privacy of clients [1]. As a result, a paradigm that possesses a centralized layout was considered in opposition to other decentralized ML frameworks [10]. The development of FedSGD has allowed an efficient communication between the server and its clients [1]. After witnessing some flaws in the FedSGD, the algorithm,

which is later optimized with FedAvg in order to remediate data imbalance, have a more understandable concept, and obtains better results. However, the utilization of FedSGD and FedAvg has shown that FL is mostly vulnerable to model poisoning from clients' updates. Thus, these two FL algorithms have been improved to create more robust aggregation methods, such as α -trimmed mean, Byzantine-Robust Distributed Learning, coordinate-wise median, etc. [11], [12], [13].

Zhu et al. [14] introduced a new scheme called deep leakage from gradient. With FL clients having to derive gradients locally, they designed a method that entirely recovers the contributed data shared by its gradient. In the same perspective of insuring the integrity of the FL model to withstand any corruption, the existing works in [5] and [15] showed that weight-clipping and noise addition could be very effective. They established a threshold M for the update and clipped them for aggregation. For the noise addition, a Gaussian noise is appended. Following that further, a per-client learning rate was developed by [16] to assign each client its own learning rate. Later, Ozdayi et al. [6] expanded the SignSGD method [17] to a robust learning rate (RLR) method, which applies a sign aggregation on the server's learning rate at each round. More recently, Fang and Ye [18] developed a heterogeneous FL (RHFL) that deals with label noise and performed FL in a single framework.

Data Heterogeneity: With the upsurge of data sharing from the massive participation of IoT systems in FL, data heterogeneity is an essential but often ignored factor in the system's robustness. In fact, the existing studies in the literature often overlook how much the heterogeneous nature of their data affects the obtained results. Some authors have mentioned it but did not go as deep as they should to fully evaluate its impact and use it to improve FL condition to withstand its vulnerabilities. For example, Zhao et al. [19] highlighted the impact of data heterogeneity and conducted the case study of the client's weight in an IID setting and in a non-IID. While a dissimilarity between client's weight with and without the influence of client's statistical heterogeneity is established, their study did not provide an exhaustive explanation of data heterogeneity's impact. Similarly, Sattler et al. [20] mentioned that statistical heterogeneity could impact the performance of the learning models.

Recently, Ozdayi et al. [6] acknowledged the positive impacts of heterogeneity on the performance of models when using the RLR technique against backdoor attacks. Actually, all the results obtained in that study showed a better performance in a non-IID settings. Likewise, Zawad et al. [9] conducted the effort on how data heterogeneity has power on FL robustness and highlighted the major curses and redemption of this one. Tian et al. [21] designed a novel weight-similarity-based (WSCC) method, which categorizes clients in groups based on their data distribution. The WSCC is a sophisticated but effective method; however, the main focus of this method is the client's weight instead of the heterogeneity impact of the clients. In spite of all of these related works, the immanency of exploiting statistical heterogeneity leaves us with no other choice, but going beyond identifying its impacts. We also see the necessity of utilizing it as a remedy for FL

issues, such as client selection, data imbalance, and lack of participation, among others.

Cooperative Game Theory: In its most basic definition, game theory can be explained as the mathematical study of conflict models and cooperation between intelligent rational decision-makers. For Nash's program, cooperative game theory is to establish cooperative solutions in such a manner that each given name belonging to a cooperative solution represents a Nash equilibrium of a certain number of cooperative transformations in the game [22]. In a more elaborated way, the notion of cooperative game theory can be defined as the study of game players' behaviors within coalitions. Using a derivation of this cooperative game theory named after the brilliant Nobel prize economist Lloyd Shapley, the concept of the Shapley value has brought disruptive changes in FL. The approach in [23] is to evaluate the role of each player in the game by considering the player's participation in all the possible coalitions. As a result, the Shapley value is the payoff of an average weight of the overall player's participation in the coalition game [22].

In FL, the Shapley value concept has been widely used to improve clients' contributions. To this end, a characteristic function is assigned to all existing coalitions composed of a set of players. A payoff is attributed to players in each coalition game. In order to set up a method of participant contributions measurement, Wang et al. [24] provided a simple but powerful technique to fairly compute the group feature importance using Shapley values. Nagalapatti and Narayanam [7] developed the Shapley value-based federated averaging (S-FedAvg) algorithm to allow servers in FL to select clients based on their relevancy. More recently, Liu et al. [25] proposed the guided truncation gradient Shapley (GTG-Shapley) method that addresses the challenge of enormous computation cost that previous Shapley values technique were raising. What the GTG-Shapley is doing is reconstructing FL models shared by gradient updates for Shapley values calculation rather than altering clients' combination repeatedly in training. Notwithstanding the extensive studies and progress conducted about cooperative game theory in FL, the impact of data heterogeneity remains an unsolved issue. Thus, implementing FL schemes that make use of statistical heterogeneity to reduce weakness and augment model performance.

III. PRELIMINARIES

In this section, we briefly elaborate on the baseline of our work. The key notations used in this article are listed in Table I. The cooperative training of the model between server and clients in FL follows a multiround cycle until model aggregation is reached. In a private and efficient communication, the server initiates a model with a classifier $f(w)$ that has a weight (w) distributed to the relative clients. In the computation of this model, the function $f(w)$ is subject to a minimization of the loss reflected in the following:

$$\arg \min_{w \in \mathbb{R}^d} f(w) = \frac{1}{k} \sum_{k=1}^k f_k(w). \quad (1)$$

TABLE I
TABLE OF NOTATION

(K)	Number of clients
B	Batch size
E	Number of local epoch
n_k	Total number of samples of a client k
γ	Forgetting coefficient
η	Learning rate
β	Differential privacy
ϵ	Step Size
C	Clients' fraction
(w)	Weight or global parameter vector
$f_k(\cdot)$	Loss function of a client k
$f(\cdot)$	Global Loss function
$l(\cdot)$	Loss function for an instance
w_t^k	The weight parameter for node k
w_{t+1}^k	FedAvg weight for the new round
D, D_k	Global and Local datasets
θ	Cosine distance
δ	Heterogeneity level threshold
λ	Client participation threshold
HI	Heterogeneity Index

Considering the loss function $f_k(w)$ of the k th client, the node $k = 1, 2, \dots, K$ has access to a portion of the overall client's data set n_k

$$f_k(w) \doteq \frac{1}{n_k} \sum_{j=1}^{n_k} l(\mathbf{x}_j, \mathbf{y}_j, w) \quad (2)$$

which displays the empirical risk minimization f_k with the sum of specific instance l and the vectors $(\mathbf{x}_j, \mathbf{y}_j)$, respectively, being the feature vector and label of the j th sample. It is important to note that the total number of samples in each node k local data set is $n_k = |D_k|$ and the entire data set is designated as $|D| = \sum_{k=1}^n |D_k|$ as expressed by

$$f(w) \doteq \frac{\sum_{k=1}^K |D_k| f_k(w)}{|D|}. \quad (3)$$

A protocol in FL begins with a sampling of a subset of agents S_t by a server, which also sends the model weights w_t to the same agents at a round t . Then, there is initialization and training of the transmitted model performed by the k th agent to have the resulting weight w_t^k using some iterative method such as stochastic gradient descent (SGD). A local update computation $\Delta_t^k = w_t^k - w_t$ is performed by the agent before sending it back to the server. In return, the server will proceed to the computation of the next round's weights via the aggregation function $\mathbf{g}: R^{|S_t| \times d} \rightarrow R^d$ to obtain the new weight $w_{t+1} = w_t + \eta \cdot \mathbf{g}(\{\Delta_t\})$. Here, η is the learning rate of the server and $\{\Delta_t\}$ is equal to $\cup_{k \in S_t} \Delta_t^k$. Typically, in FL, this mentioned aggregation rule is referred as the FedAvg expressed as follows:

$$w_{t+1} = w_t + \eta \frac{\sum_{k \in S_t} n_k \cdot \Delta_t^k}{\sum_{k \in S_t} n_k}. \quad (4)$$

Concretely, the server manages a validation data set on which previous protocols will loop to the point. The appropriate accuracy and the proper statistical heterogeneity are obtained for the model.

TABLE II
PARAMETERS OF THE PRELIMINARY EXPERIMENT

Parameters	Values
R	30
K	10
E	1
B	120
β	0.01
C	1
γ	0.75
ϵ	1.3

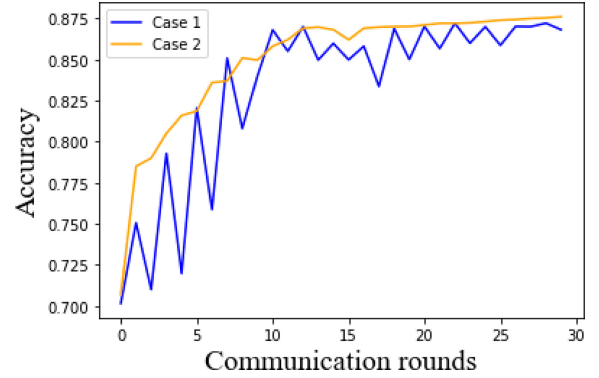


Fig. 2. Performance of FedAvg algorithm applies FEMNIST data set, where Case 1: six clients with applicable and four clients with nonapplicable data and Case 2: ten clients with applicable data.

Now, to motivate our incentive design method that goes beyond existing FL works, we undergo a short preliminary experiment that reveals its relevancy. The parameters are shown in Table II. This prelude experiment shows the necessity of an intelligent client selection, the impact of non-applicable data, and the importance of a framework that incites quality data contribution on the central model using FedAvg. Giving two case scenarios: 1) some clients have nonapplicable data (even digits) and 2) all clients have applicable data (odd digits). Here, the goal of the central server is to build a neural network that categorized all the handwritten digits that are oddly labeled $\{1, 3, 5, 7, 9\}$ using the federated extended MNIST (FEMNIST) data set.

As a result, the first curve in Fig. 2 represents case 1 and shows a very perturbed graph with numerous oscillations. The result in the first case scenario shows an unstable model in the case where there are some (4 out of 10) that are non-applicable or irrelevant to our classification; whereas, in the second case, we observe a steadier convergence model. The result for case 2 shows that all clients are contributing relevant data (10/10). The accuracy performance of the second case scenario in Fig. 2 shows a smooth training curve that reflects the stability of the model accuracy. Consequently, the necessity of managing clients with applicable data in FL is crucial. Additionally, this prelude experimentation has validated that noises added by data heterogeneity highly influence the evolution of the accuracy curves during model training.

Algorithm 1 FedAvg—Server Side

```

1: Initialize model with global weight  $w_t$  at  $t$ 
2: Distribute  $w_t \rightarrow k \in [K]$ 
3: while Clients  $\nrightarrow$  converge do
4:   Receive local updates  $w_t^1, w_t^2, w_t^3, \dots, w_t^k$ 
5:   Select random benchmark  $S_t \leftarrow [1, n] \in [K]$ 
6:   for  $k \in \{1, 2, 3, \dots, n\}$  do
7:     Update  $w_t \rightarrow w_{t+1}^k$ 
8:   end for
9:   Calculate the average global weight  $w_{t+1}$ 
10:  Optimize  $(w_t, w_{t+1}^k) \rightarrow w_{t+1}$ 
11:  Share model with global weight  $w_{t+1}$  at  $t + 1$ 
12: end while

```

Algorithm 2 FedAvg—Client Side

```

1: Local model update
2: Initialize  $k \in [K]$ ,  $E = \#$  of epochs,  $B \leftarrow$  a subset of  $S_t$ ,
3: while local epoch  $e \in [1, E]$  do
4:   Receive  $w_t$ 
5:   for batch  $e \in B$  do
6:      $w_t - \eta \nabla f_k(w_t^k, b) \rightarrow w_t^k$ 
7:   end for
8: end while
9: return  $w_t^k$  to server

```

IV. INCENTIVE DESIGN FOR HETEROGENEOUS CLIENT SELECTION

We, now, discuss the details of our incentive design approach. In our first section, we deeply elaborate our baseline approach which is FedAvg. Although FedAvg is becoming less and less a good barometer for FL efficiency in the literature, we are still using it as a landmark to craft the IHCS approach. We, then, introduce in the following section the HIC method. With some computations and an algorithm, we explore the mechanism behind our clustering method. Finally, we present our approach and explain the rationality of each step of our IHCS design.

A. Federated Averaging Approach

The increase in high-performing FL schemes has given the impression that the basic FedAvg is obsolete. However, scrutinizing the methodology of FedAvg leads us to simple but powerful new algorithms that ameliorate robustness in FL. Algorithm 1 shows the process of FedAvg on the server side. We can see at a round t , the server shares the model with the clients. Lines 3–8 display a *while* loop that is maintained until convergence is met.

Algorithm 2 represents the computation of the FedAvg on the client side. The k th client performs a local model update of the initial model received from the server. We can, respectively, see the server and client side of the FedAvg. This decentralized approach of having a server sends a model to clients to train privately is the essence of FL.

Without derogating from this essence, we design a more advanced method that helps overcome FedAvg challenges. Actually, FedAvg works better in an IID setting. In a non-IID

Algorithm 3 HIC—Server Side

```

1: Initialize model with global weight  $w_t$  at  $t$ 
2: Set threshold  $\delta = 2\%$  for  $HI(c)$  Eq. (5)
3: Distribute  $w_t \rightarrow k \in [K]$ 
4: while Clients  $\nrightarrow$  converge do
5:   Receive local updates  $w_t^1, w_t^2, w_t^3, \dots, w_t^k$ 
6:   Select random benchmark  $j \in [1, n]$ 
7:   Empty distance list  $\theta = []$ 
8:   for  $k \in \{1, 2, 3, \dots, n\}$  do
9:     Calculate  $\theta_k = \text{CosineDistance}(HI_t^k, HI_t^j)$ 
10:    Update  $\theta_k$  to  $\theta$ 
11:   end for
12:   Cluster  $List = \text{AffinityPropagation}(\theta)$ 
13:   for  $l_i \in List$  do
14:     Aggregate partially for nodes with  $l_i$  for  $w_{t+1}^{l_i}$ 
15:     Send aggregated  $w_{t+1}^{l_i}$  to node in  $l_i$ 
16:   end for
17:   Calculate the average global weight  $w_{t+1}$ 
18:   Update  $w_t \rightarrow w_{t+1}$ 
19: end while

```

setting, FedAvg can be unsuccessful in reflecting an accurate global weight. Our IHCS is producing a global weight model in an environment that reduces negative interference created by data heterogeneity.

B. HIC Approach

The HIC is a mechanism within the IHCS that allows the training of a model inside a cluster that contains clients of the same heterogeneity level. The HIC approach uses a simple but effective way to dynamically cluster clients, which lead to the creation of a high performing and resilient system. Here, we use AP techniques to prompt the distribution of model weights into categories using a threshold. To begin with, we proceed to a quantification of the impact level of data heterogeneity. We calibrate the heterogeneity index [9]

$$HI(c) = 1 - \frac{1}{C_{\max} - 1} \cdot (c - 1) \quad (5)$$

where c allows the adjustment of the number of classes per agent, and $C_{\max}$ is the overall number of classes within the data set. Data normalization is performed with 0 being the lowest level of data heterogeneity and 1 being the highest.

In Algorithm 3, the HIC follows a similar process as in a typical FL. Prior to the distribution of the global weight to the selected clients, this one is first initialized. At round $t = 0$, the global weight parameter is initialized as w_0 , a threshold δ is set, and the model is shared among selected clients k (Line 1–3). On the client's side, upon receiving the initial weight, the HI is computed using (5) and assigned to the k th client's model. Then, the client k trains the received model via SGD and ends up with a new weight w_0^k . After a computation of its update, the client sends the updated weight $\Delta_0^k = w_0^k - w_0$ and its HI back to the server. On the server side, we have the iterative aggregation in which the clustering process based on the weight and HI is executed. Finally, the

average global weight is calculated and updated for new client selection.

Clients with similar heterogeneity indexes are clustered together. Then, inside each cluster, nodes collaboratively train distinct models with the standard FedAvg algorithm. As shown in line 9, (5) is associated with cosine distance metrics to compute the cosine distance of each subpopulation of the data set. A heterogeneity index value is associated with other parameters such as the cosine similarity θ of the node k . θ_k is appended to θ after each iteration when $k \in \{1, 2, 3, \dots, n\}$. Subsequently, to effectively estimate the number of clusters and properly spread them, we use a dynamic clustering mechanism. Line 12 indicates the use of AP clustering [26]. The AP clustering algorithm is able to determine the number of target clusters without a predefined value, which offers a great benefit for FL with an unknown number of clients before the training process.

C. IHCS Approach

In a more inclusive and exhaustive way, the IHCS approach proposes a more sophisticated version of FedAvg with the inclusion of the HIC approach to create a robust FL framework. Recall that our preliminary experiment in Section III has shown that data heterogeneity can contribute to the perplexity of balancing and monitoring data during model training. Additionally, with the massive participation of smart devices with high scalability, large variance of network bandwidth among clients, etc., the FL is becoming more exposed to a heterogeneous environment. All these mentioned factors create problems, such as overfitting in the result, distribution skewness issues, etc. Kairouz et al. [27] divided distribution skewness into two problems that are label distribution and feature imbalance. Now, the main objective of this approach is to come up with a method that minimizes the degradation of the model performance. Thus, on the server side, we proceed to minimize the loss function that allows the optimization of the global weight symbolized by

$$w_{t+1} = \arg \min_{w \in \mathbb{R}^d} \left\{ f_k(w_t^k) \right\}. \quad (6)$$

On the client side, there is a computation of the received weight w_{t+1} after each iteration by the k th client to w_{t+1}^k . The locale update weight by each client k is illustrated by

$$w_{t+1}^k = \left(w_t^k - \eta \nabla f_k(w_t^k) \right). \quad (7)$$

Here, $\nabla f_k(w_t^k)$ represents the gradient for w_t^k of the loss function $f(\cdot)$. One of the main contributions of the IHCS method is to create an intelligent client selection method to enhance robustness in FL. IHCS is applied on the aggregated server and at each round of communication with clients, the server tries to allocate what we call a level of recognition of these clients. This mechanism of client recognition consists of aggregating the value of importance of each client in different layers.

We compute the recognition value α represented by

$$\alpha_t^k = SV_k(v) \quad (8)$$

where l is the layer number and $SV_k(v)$ is the Shapley value of the client k and N is the permutation by $\pi \in \Pi(N)$ and C_π

Algorithm 4 IHCS

```

1: Initialize the Recognition level or Shapley aggregation
    $\alpha_t^k = \{\alpha_t^{l_1}, \alpha_t^{l_2}, \dots, \alpha_t^{l_k}\}$  where the layer  $l \in [1, k]$ 
2: Distribute  $w_t \rightarrow k \in [K]$ 
3: while Clients  $\neg$  converge do
4:   Receive local updates  $w_t^1, w_t^2, w_t^3, \dots, w_t^k$ 
5:   Select random benchmark  $j \in [1, n]$ 
6:   Empty distance list  $\theta = []$ 
7:   for  $k \in \{1, 2, 3, \dots, n\}$  do
8:     Set a threshold  $\lambda$  for the participation of each node
      $\text{part}_{t+1}^k = \gamma \cdot \text{part}_t^k + \epsilon \alpha_k \left[ \frac{\log(w_t - w_t^k) + 1}{\log(w_{t+1} - w_t) + 1} + \beta \mathcal{N}(0, \sigma^2) \right]$ 
9:   Calculate  $\theta_k = \text{CosineDistance}(HI_t^k, HI_t^j)$  with
      $\lambda = 2\%$ 
10:   Update  $\theta_k$  to  $\theta$ 
11: end for
12: Cluster  $List = \text{AffinityPropagation}(\theta)$ 
13: for  $l_i \in List$  do
14:   Aggregate partially for nodes with  $l_i$  for  $w_{t+1}^{l_i}$ 
15:   Send aggregated  $w_{t+1}^{l_i}$  to node in  $l_i$ 
16: end for
17: Calculate the average global weight  $w_{t+1}$ 
18: Update  $w_t \rightarrow w_{t+1}$ 
19: end while

```

represents the coalition of all previous clients. The Shapley value is formulated as follows [28]:

$$SV_k(v) = \frac{1}{|N|!} \sum_{\pi \in \Pi} [v(C_\pi(k) \cup k) - v(C_\pi(k))]. \quad (9)$$

IHCS establishes an incentive mechanism for client participation. Thus, client selection is not performed randomly within the IHCS system. Indeed, a method of calibrating the importance of each client for the targeted model is one of the most important key factors in our approach. After quantifying the recognition level at each round, we proceed to the calculation of the participation (part_t^k) of each client. The calculation of part_t^k depends on the α_t^k , which relies on the aggregated parameters previously obtained. IHCS, using (10) below, quantifies the influence level of each client k during model training. Now, the probability of selection of the client will be based on their participation level. At a round $t + 1$, the part_{t+1}^k is expressed as follows:

$$\text{part}_{t+1}^k = \gamma \cdot \text{part}_t^k + \epsilon \alpha_k \left[\frac{\log(w_t - w_t^k) + 1}{\log(w_{t+1} - w_t) + 1} + \beta \mathcal{N}(0, \sigma^2) \right]. \quad (10)$$

Here, β is the differential privacy during the transmission process of the parameters from clients to the aggregated server. Prior to that, β is multiplied by the random vector $\mathcal{N}(0, \sigma^2)$, as shown in (10). This randomization is essential to maintain the privacy of clients' local data.

Algorithm 4 illustrates the IHCS process line by line. Initially, the server set the recognition level before distributing the weight to the selected clients. Starting in line 3, the

TABLE III
STATISTICS OF DATA SETS IN LEAF

Dataset	Number of Users	Total Samples	Samples per user	
			mean	stdev
CelebA	9,343	200,288	21.44	7.63
CIFAR-10	200	120,200	2.06	3.62
FEMNIST	3,550	805,263	226.83	88.94
Sent140	660,120	1,600,498	2.42	4.71

IHCS algorithm follows the same process as HIC by running a *while* loop until convergence is reached. Lines 7–12 outlines the *for* loop procedure that set the threshold for each client's participation and the calculation of part_{t+1}^k . The remaining part of Algorithm 4 is similar to Algorithm 3 described in Section IV-B.

V. PERFORMANCE EVALUATION

In this section, we conduct extensive experimentation that confirms the efficacy of the IHCS that deals with the issues in FL. In the following, we begin with the layout of our experimental setup, including the various parameters and other settings used. Then, we present and evaluate our empirical results.

A. Methodology

Prior to the deployment of our IHCS scheme, we scrutinize some data sets that are more adequate to our non-IID setting, which is an appropriate approach [29]. To better replicate the data heterogeneity feature of our client, we use LEAF², an open-source for FL bench-marking LEAF [30] that provides six data sets that a number of state-of-the-art works in the literature have used to better simulate data heterogeneity. After developing a pre-existing untrained model from the LEAF open-source base, we implement our neural network using TensorFlow 2.x platform to implement and conduct experimentation of our approach. In Table III, we have the statistics of four LEAF's data sets, such as CelebA, which is an image data set built on a large scale of celebrity faces. CelebA has two classification tasks (smiling and not smiling). Additionally, the FEMNIST data set is an image-based data set of handwritten digits/characters with 62 distinct classes. Finally, we have the *Sentiment140* (Sent140) data set, which is a generated sentiment analysis data set from Twitter users with a positive or negative class. To further experiment the effectiveness of the IHCS, we apply our approach on the Canadian Institute for Advanced Research, ten classes (CIFAR-10) data set which contains of 6000 32×32 color images [31]. The statistics of each data set are shown in Table III.

The following experimental setup further allows a comparative evaluation of the performance improvements of our IHCS approach and some other FL methods. First, we reproduce the same data distribution as [30], which is a non-IID one. We partition the data as follows: 60% of the data for training and 40% for validation. For the CelebA, 10% of the overall users are subsampled with the rate of 0.001 and a batch size of 5.

²LEAF: <https://github.com/TalwalkarLab/leaf>.

TABLE IV
DATA DISTRIBUTION FOR FEMNIST

Cluster #	Set of K	Set of Labels	$HI(c)$ Interval
1	{1, 2, 3, 4, 5}	{0, 1, 2, 3, 4}	$0 \leq HI \leq 0.25$
2	{6, 7, 8, 9, 10}	{5, 6, 7, 8, 9}	$0.25 < HI \leq 0.5$
3	{11, 12, 13, 14, 15}	{0, 2, 4, 6, 8}	$0.5 < HI \leq 0.75$
4	{16, 17, 18, 19, 20}	{1, 3, 5, 7, 9}	$0.75 < HI \leq 1$

For the FEMNIST experiment, we subsample 5% of the data. The model used for both CelebA and FEMNIST is the same with two convolutional neural network (CNN) layers and two dense layers. The used learning rate for the FEMNIST experiment is 0.004 for the FedAvg and 0.006 for the batch size of 5. We use four CNN and two dense layers with a learning rate of 0.0005 and a batch size of 10 for CIFAR-10. For each class within the ten classes of the CIFAR-10, there are 6000 images, of which we partition it into 5000 images for training and 1000 testing images per class. For the Sent140 experiment, the logistic regression model is used for the words and a learning rate of 0.0003.

For the other hyperparameters, we conserve the ones used in the preliminary experiment displayed in Table II. We first replicate the preliminary experiment by running on the FEMNIST data set the FedAvg, the standalone HIC, and the IHCS without the integration of the HIC that we name improved fedavg client selection (IFCS)³ to see whether or not the IHCS needs clustering. The obtained results in Fig. 3 illustrate the performance of both the accuracy and loss functions of these three methods, showing the need for clustering. The obtained results in the following section can validate the higher performance of our IHCS approach compared with some other existing FL methods.

Next, we incorporate the HIC system into the IHCS algorithm to create dynamic clusters based on clients' heterogeneity level. We then carry out another experiment with our new version of IHCS that reflects the implementation of Algorithm 4. Almost all the settings of this new experiment are the same as the previous one with the exception of the number of clients k . In this case, we set $k = 20$ to allow each cluster to have five nodes and set the data distributions as shown in Table IV.

We evenly partition the data to each node with a distributed data ratio of 20% per node. Then, using (5), we create four clusters with a heterogeneity level interval of 25% increase as reflected in Table IV. The performance result for each cluster is displayed in Fig. 4 in Section V-B.

Another experiment conducted in this study is the resiliency of our approach against adversaries. In order to thoroughly test our system's robustness by using a similar attack approach used by [32], which uses systematic trigger patterns in a decentralized setting to conduct attacks. It is important to note that the decentralized setting and the way that the server operates with clients in FL makes the system strictly vulnerable to model updates. Thus, the most utilized attacks in FL are backdoors. This type of attack is done in a stealthy way

³IFCS: a premature version of IHCS that does not incorporate a heterogeneity-based clustering system.

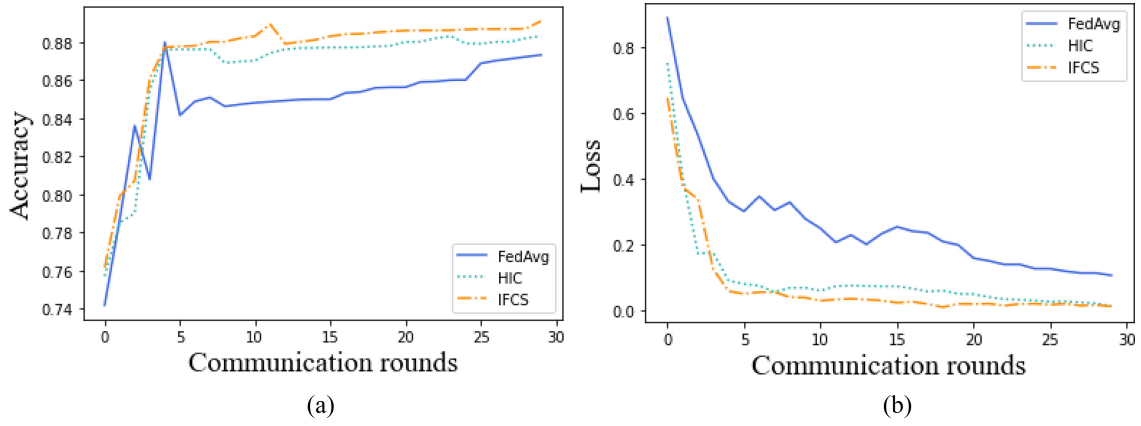


Fig. 3. Performance of FedAvg, HIC, and IFCS (FEMNIST). (a) Comparison of the accuracy. (b) Comparison of the loss function.

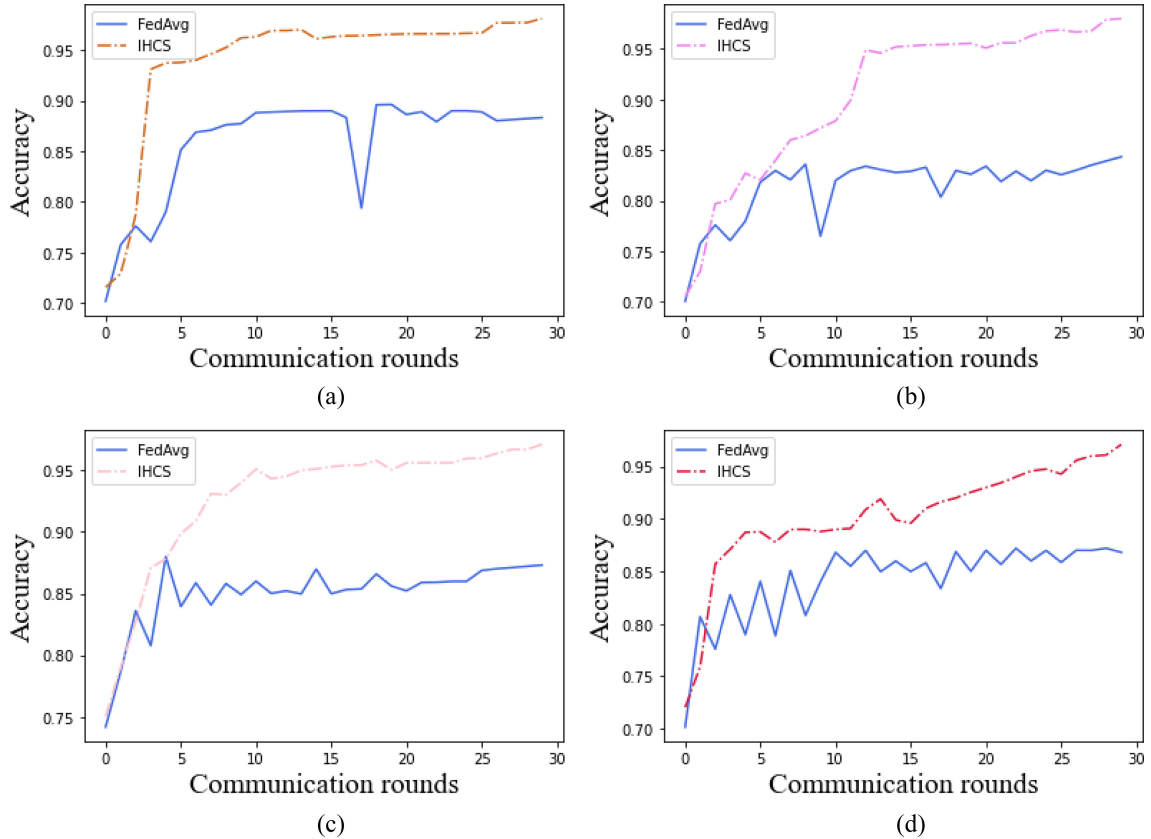


Fig. 4. Performance of FedAvg versus IHCS in clusters for FEMNIST. (a) Cluster 1. (b) Cluster 2. (c) Cluster 3. (d) Cluster 4.

by exhibiting the backdoor after poisoning the model during the update. Adversaries substitute the model weight update shown in (7) with a malicious one. To simulate some backdoor attacks toward it and measure their success rate, we establish a system of measurement of the injection success rate (ISR) for each attack on a cluster with a specific heterogeneity level. In this experiment, we consider an injection attack successful when the classified result matches the attack's target. As a result, the ISR is evaluated relative to the $HI(C)$ average of each cluster [9]. We deliberately apply a trigger on half of the data of each data set. For instance, we include a trigger string on 50% of the Sent140 for the natural language processing (NLP) and append additional patterns on

50% of the samples in the FEMNIST, CIFAR-10, and CelabA images.

B. Results and Analysis

Here, we present the acquired results from our experimentation phase above. After running the first experiment of each of the IFCS, the standalone HIC, and the FedAvg on the FEMNIST data, we evaluate the performance of these algorithms in Fig. 3, which reflects the convergence of the accuracy and the loss function. In Table V, we can see the comparative results of the FedAvg, HIC, and IFCS performances.

TABLE V
IFCS ACCURACY COMPARISON BY % IN FEMNIST

Round	FedAvg	HIC	IFCS	Improvement
1	78.67	81.35	81.40	2.73
5	83.59	86.95	88.56	4.97
10	80.77	82.92	88.76	7.99
20	83.05	87.27	88.9	5.85
30	83.16	89.71	90.02	7.04

TABLE VI
IHCS ACCURACY COMPARISON BY % IN FEMNIST

Round	FedAvg	HIC	IFCS	IHCS	Improvement
1	78.67	81.35	81.40	86.1	7.43
5	83.59	86.95	88.56	89.8	6.21
10	80.77	82.92	88.76	91.8	11.03
20	83.05	87.27	88.9	94.5	11.45
30	83.16	89.71	90.02	97.5	14.34

We can see a net improvement in the premature version of our IHCS approach. Nevertheless, we would like to go beyond FedAvg and have an IHCS version with performance comparable to the existing state-of-the-art schemes. Thus, we improve our approach by implementing our initial HIC method within the IFCS. This approach allows the dynamic clustering of the various clients by heterogeneity level. The obtained results are displayed in Table VI.

Another issue that our IHCS approach brings a solution to is data imbalance. The results shown in the preliminary experiment confirm that nonapplicable and heterogeneous data deeply affects the balance of the data. In fact, there is a contrast in the smoothness of the accuracy graphs of cases 1 and 2 in Fig. 2. On the other hand, we witness a steadier performance curve in all experiments we run. For instance, Fig. 4(a)–(d) displays the performance of different cluster under both FedAvg and IHCS. The stability of the accuracy curves in the results confirm that the IHCS contributes to minimizing data irrelevancy that negatively affects FL performance. As the heterogeneity level increases in different clusters, we observe a net augmentation of the perturbation in the FedAvg curves; whereas, the IHCS performance curves are barely perturbed by the change in heterogeneity level.

Fig. 4 illustrates the increase of the accuracy performance of different clusters with respect to $HI(c)$ intervals in Table IV. We observe that the heterogeneity levels have more impact on the FedAvg convergence of accuracy functions than the IHCS. In fact, there is a significant growth in the performance in all four clusters. In addition, we notice remarkable stability of the IHCS performance graphs despite the increase of the $HI(c)$ level. This shows the efficiency of our algorithm against the negative impact of statistical heterogeneity, data irrelevancy, etc. Also, as reflected in Fig. 5, the mean curve of the IHCS shows an increment of more than 14% at the 30th round.

In our experiments, we show that our IHCS approach significantly overcomes the issue of privacy to bring higher performance. There are a number of efforts on FL that provide solutions with strong privacy guarantees [5], [7], [14]. In comparison to these state-of-the-art works, the IHCS brings a relatively higher performance guaranteeing the privacy of

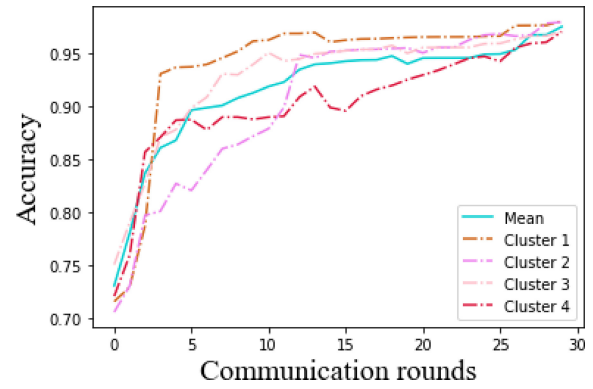


Fig. 5. Mean curve of performance of IHCS in clusters for FEMNIST.

clients' data as reflected in Fig. 5. The improved performance of the IHCS can be explained by the smart client selection method of this model and its ability to segregate model training based on heterogeneity level. Using the cooperative game theory, clients are selected with respect to both their participation value $part_i^k$ and their heterogeneity level HI_i^k . In Fig. 5, we can see the average curve of the accuracy performance of different clusters above the stand-alone performance of both FedAvg and HIC accuracy curves. This performance is also higher than the ones produced by more sophisticated FL schemes, such as FedProx [33], FOLB [34], and robust heterogeneous federated learning (RHFL) [18]. It is important to mention that our IHCS approach does not “miraculously” provide greater performance but utilizes the shared gradients from clients' local data. It also intelligently selects client and uses a technical process to segregates model's weight by heterogeneity level. Similarly, in Fig. 3(b), we can observe the impact of inhibiting data heterogeneity with a lower loss of the average IHCS curves.

After showing the robustness of our approach on the performing aspect, we present the acquired results of the series of backdoor attacks against our IHCS mechanism. We first train the data sets on standard FedAvg associated with the IHCS algorithm. Injection attacks are applied to half of the clients during each round. The selected weights are clustered based on their level of heterogeneity. All evaluations are executed 10 times for each cluster on all three data sets and the ISR after each attack is measured.

Fig. 6 is a box-and-whisker plot of the attack results. Our observation from the ISR versus Heterogeneity figures shows that those attacks are less successful between the heterogeneity level of 0.3 and 0.6 for the CelebA data set in Fig. 6(a). For the attacks on the FEMNIST data set, we see that the ISR gradually decreases as the level of heterogeneity approaches 0.6 before increasing as it gets closer to 1. The average of ISR on the CIFAR-10 is between 0.74 to 0.3 as displayed in Fig. 6(b). We also notice the attacks performed on the CIFAR-10 under IHCS are lower for heterogeneity level of 0.4 and 0.6 which result in a success rate 0.4 and 0.3, respectively. Finally, the results of performing attacks on the Sent140 display a high ISR close to 1 for the cluster with a heterogeneity level of 0.1 and a lower ISR of 0.5 at a heterogeneity level equal to 0.9. The box-and-whisker plots show that our

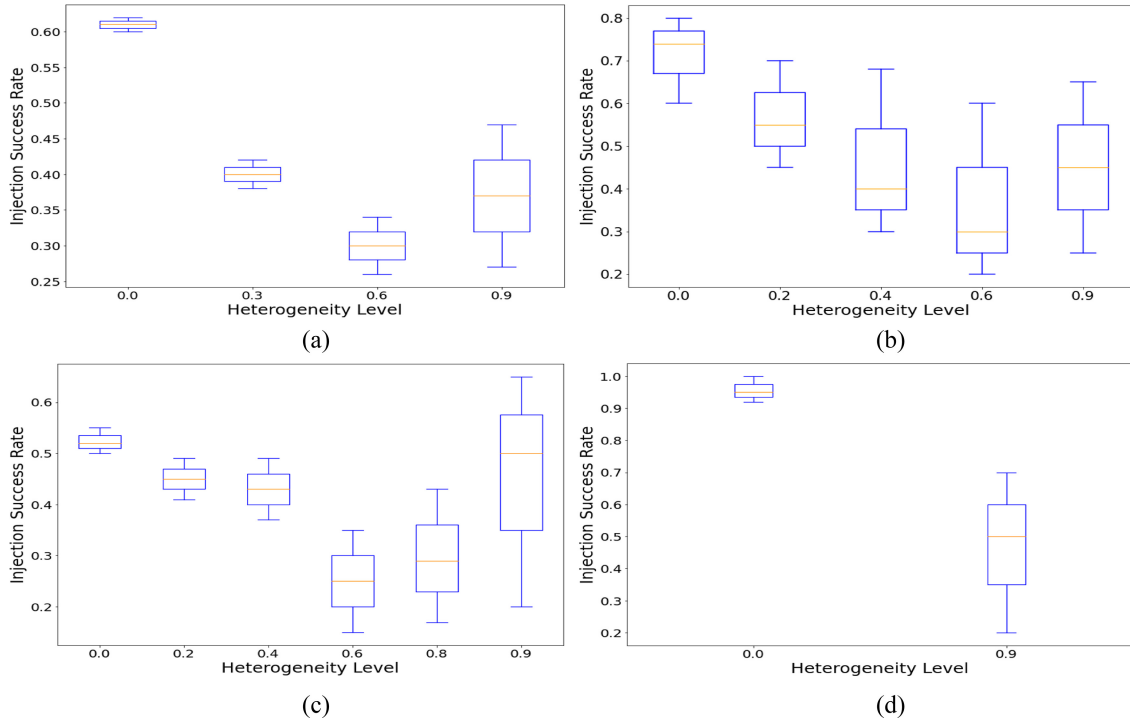


Fig. 6. ISR versus heterogeneity level. (a) CelebA. (b) CIFAR-10. (c) FEMNIST. (d) Sent140.

attack effectiveness significantly reduces as heterogeneity is average or close to 0.6. This higher resiliency that we observe at Heterogeneity Level ≈ 0.6 can be explained by the complexity of attacks to foresee the results and a relatively high performance of the IHCS after the 5th round. We also note that the thickness of boxes in almost every cluster for all data sets is bigger as heterogeneity increases in Fig. 6. The results confirm our initial hypothesis about the efficacy of attacks as the level of heterogeneity increases. It also validates the resiliency of our IHCS scheme against external attacks.

Moreover, the IHCS offers a framework that promotes client contributions. In the mentioned existing works, there is an existential issue of client contributions that industry applications are facing. Some existing works proposed a rewarding model to promote clients to participate more. This reward allocation model induces could help to improve the level of client contributions but shows limits as well. We witness some computing constraints as clients try to maximize rewards, a high cost of data procurement, etc. The IHCS uses a participation scaling method that helps remediate the lack of client participation. In Section IV-C, we elaborate on our incentive mechanism that induces an effective client selection, which reduces the lack of clients' contributing relevant data during model training. In Fig. 7, the level of contribution is significantly superior in the IHCS system compared to the FedAvg for all clients index. Consequently, we can say that the objective of crafting a high-performing, flexible, and secure robust FL framework is realized.

VI. DISCUSSION

In the following, we discuss the remaining issues that the IHCS approach is not covering and how to extend the



Fig. 7. Comparative results of clients participation under FedAvg versus IHCS.

framework to be more generic to other systems. Concretely, in this section, we are giving some open perspectives starting from our IHCS scheme.

A remaining issue the IHCS is not solving is the smart attacks that target label flipping. With the assumption that data heterogeneity helps to increase the success rate of these types of attacks, it would be a good idea to applying the IHCS to counter them. We can consider the implementation of our incentive design to measure the impact of benign versus malicious updates during backdoor attacks. Then, we can include a data-label standardization method during server-client communication to make our IHCS resilient against label-flipping attacks.

Additionally, in this study, we develop a framework that uses both cooperative game theory and heterogeneity to validate our work on performance deterioration and security exposure

in FL. After studying the problems of data imbalance, data relevancy, and clients' contribution, we confirm through extensive experiments the effectiveness of the IHCS against these issues. However, the IHCS is a generic framework, which can be applied in different systems other than FL. For instance, the smart client selection and dynamic heterogeneous-based clustering methods of the IHCS can be utilized to mitigate problems of edge-based computing that support real-time location in digital twin (DT) systems [35].

Furthermore, this article opens other research perspectives such as the feasibility of the IHCS on smart-world systems. Actually, the utilization of our scheme to better apprehend the scalability of IoT devices in IoT-based smart-world systems (smart grid, smart transportation, smart manufacturing, etc.) would make it more generic. Indeed, it would be interesting to explore the application of the Shapley value to the average client model's weight in SG and train these models in dynamic heterogeneous clusters. The implementation of IHCS in this particular scenario can help answer some source-efficient issues experienced in IoT systems (e.g., energy in a smart grid).

VII. FINAL REMARKS

In this article, we have proposed an IHCS scheme that mitigates performance degradation in FL due to the challenges of lacking access to clients' data, unbalanced data, and the lack of applicable data contribution from clients. These challenges negatively affect the robustness of FL. By embedding the HIC technique within the IHCS to create iterative dynamic categorization, we are able to isolate the negative impact of data heterogeneity and establish a robust FL framework that produces a model with high performance. Moreover, the proposed IHCS approach allows the decentralized server to intelligently select clients with a higher contribution in both a qualitative and quantitative manner using the cooperative game theory method. Our proposed approach, which has overcome server blindness toward FL clients, is able to filter clients with higher contributions using a smart clients' selection method, mitigate statistical heterogeneity issues, and improve the robustness in FL. The extensive experimental results confirm that the proposed IHCS approach outperforms representative schemes.

ACKNOWLEDGMENT

Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the United States Air Force.

REFERENCES

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Stats.*, 2017, pp. 1273–1282.
- [2] F. Liang, W. Yu, X. Liu, D. Griffith, and N. Golmie, "Toward edge-based deep learning in industrial Internet of Things," *IEEE Internet Things J.*, vol. 7, no. 5, pp. 4329–4341, May 2020.
- [3] W. G. Hatcher and W. Yu, "A survey of deep learning: Platforms, applications and emerging research trends," *IEEE Access*, vol. 6, pp. 24411–24432, 2018.
- [4] F. Liang, W. G. Hatcher, W. Liao, W. Gao, and W. Yu, "Machine learning for security and the Internet of Things: The good, the bad, and the ugly," *IEEE Access*, vol. 7, pp. 158126–158147, 2019.
- [5] R. C. Geyer, T. Klein, and M. Nabi, "Differentially private federated learning: A client level perspective," *NIPS Workshop*, pp. 1–7, 2017.
- [6] M. S. Ozdayi, M. Kantarcioglu, and Y. R. Gel, "Defending against backdoors in federated learning with robust learning rate," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 10, pp. 9268–9276, May 2021. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/17118>
- [7] L. Nagalapatti and R. Narayanam, "Game of gradients: Mitigating irrelevant clients in federated learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 10, 2021, pp. 9046–9054.
- [8] J. Pang, Y. Huang, Z. Xie, Q. Han, and Z. Cai, "Realizing the heterogeneity: A self-organized federated learning framework for IoT," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3088–3098, Mar. 2021.
- [9] S. Zawad et al., "Curse or redemption? How data heterogeneity affects the robustness of federated learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 12, 2021, pp. 10807–10814.
- [10] J. Dean et al., "Large scale distributed deep networks," *Adv. Neural Inf. Process. Syst.*, vol. 25, pp. 1–9, Oct. 2012.
- [11] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 118–128.
- [12] E. M. E. Mhamdi, R. Guerraoui, and S. Rouault, "The hidden vulnerability of distributed learning in byzantium," in *Proc. Int. Conf. Mach. Learn.* PMLR, 2018, pp. 3521–3530.
- [13] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. Int. Conf. Mach. Learn.* PMLR, 2018, pp. 5650–5659.
- [14] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," *Adv. Neural Inf. Process. Syst.*, vol. 32, pp. 1–11, 2019.
- [15] Z. Sun, P. Kairouz, A. T. Suresh, and H. B. McMahan, "Can you really backdoor federated learning?" *2nd Int. Workshop Federated Learn. Data Privacy Confidentiality, NeurIPS*, pp. 1–10, Nov. 2019.
- [16] C. Fung, C. J. Yoon, and I. Beschastnikh, "Mitigating sybils in federated learning poisoning," 2018, *arXiv:1808.04866*.
- [17] J. Bernstein, J. Zhao, K. Azizzadenesheli, and A. Anandkumar, "SignSGD with majority vote is communication efficient and fault tolerant," *Int. Conf. Learn. Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=BJxhijAcY7>
- [18] X. Fang and M. Ye, "Robust federated learning with noisy and heterogeneous clients," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10072–10081.
- [19] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-iid data," 2018, *arXiv:1806.00582*.
- [20] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Robust and communication-efficient federated learning from non-i.i.d. data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 9, pp. 3400–3413, Sep. 2020.
- [21] P. Tian, W. Liao, W. Yu, and E. Blasch, "WSCC: A weight-similarity-based client clustering approach for non-IID federated learning," *IEEE Internet Things J.*, vol. 9, no. 20, pp. 20243–20256, Oct. 2022.
- [22] R. B. Myerson, *Game Theory: Analysis of Conflict*. Cambridge, MA, USA: Harvard Univ. Press, 1997.
- [23] L. S. Shapley, "Cores of convex games," *Int. J. Game Theory*, vol. 1, no. 1, pp. 11–26, 1971.
- [24] G. Wang, C. X. Dang, and Z. Zhou, "Measure contribution of participants in federated learning," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, 2019, pp. 2597–2604.
- [25] Z. Liu, Y. Chen, H. Yu, Y. Liu, and L. Cui, "GTG-Shapley: Efficient and accurate participant contribution evaluation in federated learning," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 4, pp. 1–21, 2022.
- [26] B. J. Frey and D. Dueck, "Clustering by passing messages between data points," *Science*, vol. 315, no. 5814, pp. 972–976, 2007.
- [27] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends® Mach. Learn.*, vol. 14, nos. 1–2, pp. 1–210, 2021.
- [28] D. Monderer and L. S. Shapley, "Potential games," *Games Econ. Behav.*, vol. 14, no. 1, pp. 124–143, 1996.
- [29] S. Li, E. Ngai, F. Ye, and T. Voigt, "Auto-weighted robust federated learning with corrupted data sources," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 5, pp. 1–20, 2022.
- [30] S. Caldas et al., "Leaf: A benchmark for federated settings," 2018, *arXiv:1812.01097*.
- [31] A. Krizhevsky, G. Hinton, "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, ON, Canada, Rep. TR-0, 2009.

- [32] C. Xie, K. Huang, P.-Y. Chen, and B. Li, "DBA: Distributed backdoor attacks against federated learning," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–19.
- [33] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, vol. 2, pp. 429–450, 2020.
- [34] S. R. Pokhrel and J. Choi, "Federated learning with blockchain for autonomous vehicles: Analysis and design challenges," *IEEE Trans. Commun.*, vol. 68, no. 8, pp. 4734–4746, Aug. 2020.
- [35] C. Qian, X. Liu, C. Ripley, M. Qian, F. Liang, and W. Yu, "Digital twin-cyber replica of physical things: Architecture, applications and future research directions," *Future Internet*, vol. 14, no. 2, 2022. [Online]. Available: <https://www.mdpi.com/1999-5903/14/2/64>



Papa Pene received the B.S. degree from Strayer University, Washington, DC, USA, in 2019, and the M.S. degree in computer science from Towson University, MD, USA, in 2022, where he is currently pursuing the Ph.D. degree with the Department of Computer and Information Sciences.

His research interests include distributed machine learning, big data, and cybersecurity for cyber-physical systems and Internet of Things.



Weixian Liao received the B.S. degree in information engineering from Xidian University, Xi'an, China, in 2012, the M.S. degree in electrical and computer engineering from Mississippi State University, Starkville, MS, USA, in 2015, and the Ph.D. degree in computer engineering from Case Western Reserve University, Cleveland, OH, USA, in 2018.

He is currently an Assistant Professor with the Department of Computer and Information Sciences, Towson University, MD, USA. His research interests

include cybersecurity and optimization in big data applications, cyber-physical systems, and machine learning.



Wei Yu received the B.S. degree in electrical engineering from Nanjing University of Technology, Nanjing, China, in 1992, the M.S. degree in electrical engineering from Tongji University, Shanghai, China, in 1995, and the Ph.D. degree in computer engineering from Texas A&M University at College Station, College Station, TX, USA, in 2008.

He is currently a Full Professor with the Department of Computer and Information Sciences, Towson University, MD, USA. His current research interests include cybersecurity and privacy, cyber-

physical systems/Internet of Things, data science, and machine learning-driven applications.

Dr. Yu was a recipient of the 2014 NSF CAREER Award, the 2015 University System of Maryland (USM) Regents Faculty Award for Excellence in Scholarship, Research, or Creative Activity, and the 2016 USM Wilson H. Elkins Professorship.